

Design Patterns

Clément RENIERS

Table des matières

Stratégie	2
Principe	2
Exemple (Tri)	2
État	3
Principe	3
Exemple (Lecteur audio)	3

Stratégie

Principe

⇒ On dépend des classes abstraites, rajouter une stratégie ne change rien au code.

⇒ Utilisable quand on a plusieurs variantes d'un algorithme.

⇒ Au contraire du pattern State, on change de stratégie de manière explicite. Les stratégies ne sont pas liées à un état de l'application.

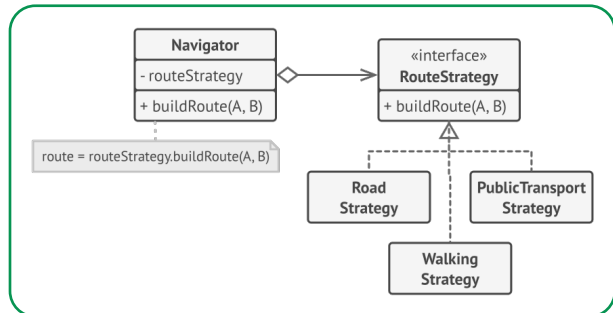


Figure 1: Utilisation simple du pattern Stratégie

Exemple (Tri)

```
1 public interface SortingStrategy {
2     void sort(int[] array);
3 }
4
5 public class SortingContext {
6     private SortingStrategy sortingStrategy;
7
8     public SortingContext(SortingStrategy sortingStrategy) {
9         this.sortingStrategy = sortingStrategy;
10    }
11
12    public void setSortingStrategy(SortingStrategy sortingStrategy) {
13        this.sortingStrategy = sortingStrategy;
14    }
15
16    public void performSort(int[] array) {
17        sortingStrategy.sort(array);
18    }
19 }
```

```
1 public interface SortingStrategy {
2     void sort(int[] array);
3     // Implement sorting logic here
4 }
```

```
1 // BubbleSortStrategy
2 public class BubbleSortStrategy implements SortingStrategy {
3     @Override
4     public void sort(int[] array) {
5         // Implement Bubble Sort algorithm
6         System.out.println("Sorting using Bubble Sort");
7     }
8 }
9
10 // MergeSortStrategy
11 public class MergeSortStrategy implements SortingStrategy {
12     @Override
13     public void sort(int[] array) {
14         // Implement Merge Sort algorithm
15         System.out.println("Sorting using Merge Sort");
16     }
17 }
18
19 // QuickSortStrategy
20 public class QuickSortStrategy implements SortingStrategy {
21     @Override
22     public void sort(int[] array) {
23         // Implement Quick Sort algorithm
24         System.out.println("Sorting using Quick Sort");
25     }
26 }
```

État

Principe

- ⇒ Sert dans le cas d'une classe qui doit avoir un comportement différent selon les états dans lesquels elle se trouve.
- ⇒ Permet d'éviter les gros blocs conditionnels
- ⇒ C'est l'état qui se charge de réaliser les actions.
- ⇒ Différence avec **Stratégie** : le comportement peut-être changé dans la durée de vie de la classe

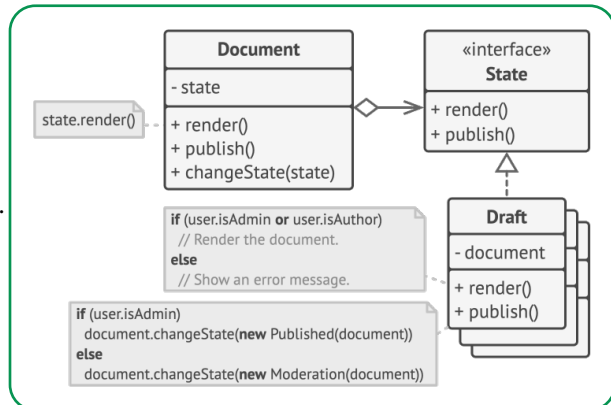


Figure 2: Utilisation simple du pattern État

Exemple (Lecteur audio)

- ⇒ On imagine un lecteur audio avec un unique bouton pour play/pause.
- ⇒ On modélise deux états : "Playing" et "Paused". Le comportement du bouton change selon l'état dans lequel on se trouve.

```
1 public class Reader {
2     private ReaderState _state = null;
3
4     public Reader(ReaderState state) {
5         this._state = state;
6     }
7
8     public void PressPlay() {
9         this._state.PressPlay(this);
10    }
11
12    public ReaderState CurrentState {
13        get { return _state; }
14        set { _state = value; }
15    }
16 }
```

```
1 public abstract class ReaderState {
2     public abstract void PressPlay(Reader reader);
3 }
4
5 public class ReaderPlayingState extends ReaderState {
6     public ReaderPlayingState() {
7         System.out.println("Reader playing");
8     }
9
10    @Override
11    public void PressPlay(Reader reader) {
12        reader.CurrentState = new ReaderPausedState();
13    }
14 }
15
16 public class ReaderPausedState extends ReaderState {
17     public ReaderPausedState() {
18         System.out.println("Reader paused");
19     }
20
21    @Override
22    public void PressPlay(Reader reader) {
23        reader.CurrentState = new ReaderPlayingState();
24    }
25 }
```